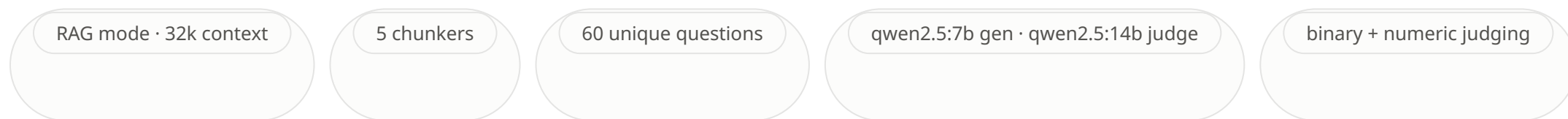


# Does chunking strategy change RAG accuracy?

Swapping LaRA's chunking step for five **Chonkie** chunkers — token, sentence, recursive, semantic, fast — while holding retrieval, generation, and judging fixed, then scoring on both a binary and a 0–10 numeric scale.



Aaditya Bhilegaonkar · 2026-08-05

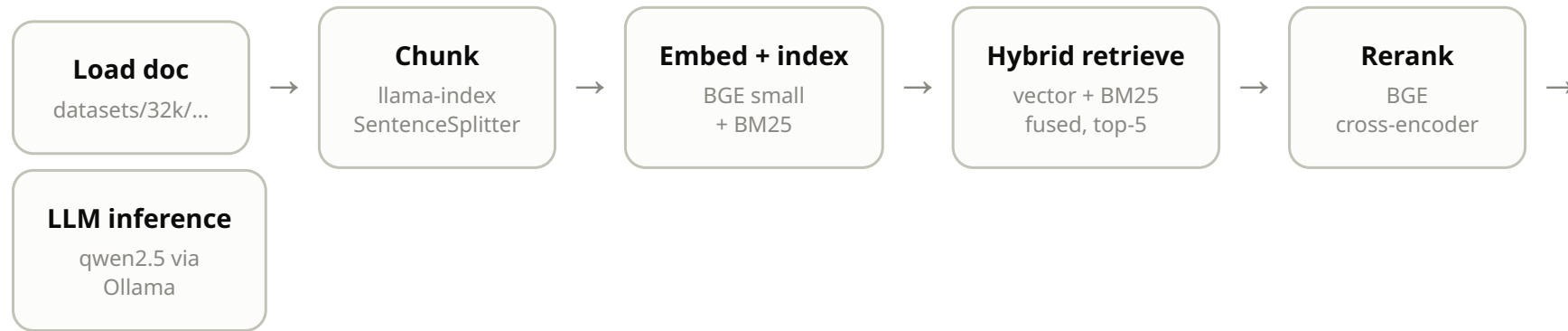
## THE ASK

# This week's job, as given

*"Build a LaRA evaluation pipeline that uses the Chonkie library directly for chunking. That's it. Don't worry about integrating with Doug's MongoDB, Langfuse, or LangGraph infrastructure. Define a single chunking function that accepts a document and a chunking algorithm as a parameter... Token, Sentence, Recursive, Semantic, Fast. Flow: load LaRA document → chunk it → embed and store → retrieve top-k → run LLM inference → score."*

Scope decision made with Aaditya before building: keep LaRA's own LLM-as-judge for scoring (both binary and the numeric scale already built for the professor's other ask) instead of semantic-similarity scoring, since the goal is to compare chunking strategies under LaRA's existing, already-understood grading — not introduce a second new variable at the same time.

# One pipeline, six steps, already built

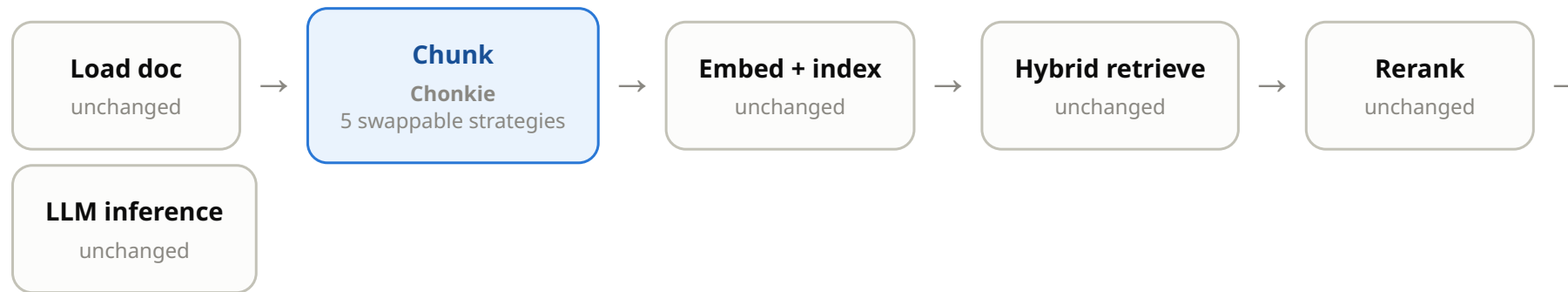


Implemented in `evaluation/eval_rag.py` (chunking) and `evaluation/search/simpleHybridSearcher.py` (embed + BM25 + fuse + rerank). This is real, working infrastructure — not something built for this task. It was chunking with one fixed strategy (llama-index's SentenceSplitter, 600-char chunks, 100-char overlap) with no way to swap it.

**The question: does the chunking step actually matter for RAG accuracy — and can we swap it out to measure that?**

## WHAT CHANGED

# Exactly one seam, everything else untouched



The hybrid searcher **re-embeds nodes itself** (`VectorStoreIndex(nodes, embed_model=...)` in `simpleHybridSearcher.py`), so the new chunking adapter only has to produce plain text nodes — it does not need to embed anything. That is why swapping the chunker is a genuinely isolated change: BM25, vector fusion, reranking, and generation see the exact same code path regardless of which chunker produced the nodes.

Not done: no MongoDB, Langfuse, or LangGraph integration, per the brief. No new vector DB — retrieval reuses LaRA's existing in-process hybrid index.

# What each one actually does

## 1 · TOKEN

### Token

Splits on a fixed character/token count, no regard for sentence or paragraph boundaries. The simplest possible baseline — closest in spirit to LaRA's original splitter.

```
chunk_size=600,  
overlap=100
```

## 2 · SENTENCE

### Sentence

Splits on sentence boundaries (. ! ? \n), then packs whole sentences into a chunk up to the size cap. Never cuts a sentence in half.

```
chunk_size=600,  
overlap=100
```

## 3 · RECURSIVE

### Recursive

Tries paragraph breaks first, then sentence punctuation, then clause punctuation, then whitespace — falling back a level only when a piece is still too big. No overlap parameter.

```
chunk_size=600
```

## 4 · SEMANTIC

### Semantic

Embeds sentences (model2vec portion-base-32M) and splits where topic similarity drops — chunk boundaries follow meaning, not fixed size. No overlap parameter.

```
chunk_size=600 (cap)
```

## 5 · FAST

### Fast

Chonkie's speed-optimized splitter — simple delimiter search with no tokenizer call per candidate boundary. Built for throughput over precision. No overlap parameter.

```
chunk_size=600
```

chunk\_size held at 600 (characters) for all 5, matching LaRA's original splitter, so the comparison isolates **strategy**, not chunk granularity. Recursive/Semantic/Fast not having an overlap knob is a real property of those strategies in Chonkie, not an oversight in this implementation.

# Five files touched, one new module

| FILE                                                             | CHANGE                                                                                                                                                                                                                           |
|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>evaluation/chunkers.py</code>                              | <b>New.</b> The adapter: <code>get_nodes(text, strategy)</code> builds the requested Chonkie chunker and wraps its output as llama-index <code>TextNode</code> s.                                                                |
| <code>evaluation/eval_rag.py</code>                              | Added <code>--chunker</code> flag; replaced the hardcoded <code>SentenceSplitter</code> + <code>IngestionPipeline</code> block with one call to <code>chunkers.get_nodes()</code> . Output filenames now carry the chunker name. |
| <code>evaluation/compute_score_llm.py</code> (binary judge)      | Added <code>--chunker</code> ; when set, reads the chunker-tagged prediction files and skips full-context (chunking doesn't apply there).                                                                                        |
| <code>evaluation/compute_score_numeric.py</code> (numeric judge) | Same <code>--chunker</code> threading, mirrors the binary judge's change.                                                                                                                                                        |
| <code>evaluation/merge_judgments.py</code>                       | Same <code>--chunker</code> threading, so the combined binary+numeric log is produced per chunker.                                                                                                                               |
| <code>evaluation/run_chunker_study.sh</code>                     | <b>New.</b> Orchestrates the full sweep: generate × 5 chunkers × 3 doc types × 4 task types, then judge (both scales), then merge, then aggregate.                                                                               |
| <code>evaluation/aggregate_chunker_results.py</code>             | <b>New.</b> Reads each chunker's binary + numeric result CSVs and builds the one final comparison table.                                                                                                                         |

## chunkers.py — the entire seam

```
def _build_chunker(strategy, chunk_size=600, chunk_overlap=100):
    if strategy == "token":      # fixed-size, no boundary awareness
        return TokenChunker(chunk_size=chunk_size, chunk_overlap=chunk_overlap)
    if strategy == "sentence":   # packs whole sentences to the size cap
        return SentenceChunker(chunk_size=chunk_size, chunk_overlap=chunk_overlap)
    if strategy == "recursive":  # paragraph → sentence → clause → whitespace
        return RecursiveChunker(chunk_size=chunk_size)
    if strategy == "semantic":   # splits where embedding similarity drops
        return SemanticChunker(chunk_size=chunk_size)
    if strategy == "fast":       # delimiter search, no tokenizer per boundary
        return FastChunker(chunk_size=chunk_size)

def get_nodes(text, strategy, chunk_size=600, chunk_overlap=100):
    chunker = _build_chunker(strategy, chunk_size, chunk_overlap)
    chunks = chunker(text)          # run the actual Chonkie chunker
    return [TextNode(text=c.text) for c in chunks]  # -> LaRA's own node type
```

One function, one signature, all 5 strategies swappable through the same call — exactly as specified in the ask. Nodes carry only .text; embeddings are computed downstream by the existing hybrid searcher.

## eval\_rag.py — the only lines that changed

```
parser.add_argument('--chunker', default='sentence', choices=chunkers.STRATEGIES)
...
# was: SentenceSplitter(...) + IngestionPipeline(...).run(documents=[doc])
nodes = chunkers.get_nodes(eg_txt, chunker)      # <- the entire replacement

searcher = SimpleHybridSearcher(config, nodes)   # unchanged: same hybrid search call
search_nodes = searcher.process(eg['question'])

# output filename now carries the chunker, so the 5 strategies don't collide:
output_path = f'./prediction/{eval_model}/rag_preds_{eval_model}_{chunker}_{context_length}_{context_type}_{query_type}.jsonl'
```

Everything below `searcher = SimpleHybridSearcher(...)` is byte-for-byte the same code that ran before this task — hybrid retrieval, reranking, and the LLM call never needed to know which chunker produced their input.



## Binary judge: True / False

LaRA's original judge (`compute_score_llm.py`). One LLM call per question, forced to a single word.

```
prompt = f"""I will provide you with a question and its groundtruth answer, as well as
an answer from an AI assistant. You need to judge whether the AI assistant's
answer is correct based on the groundtruth answer. If it is correct, you
should only output True; if it is incorrect, only output False."""

...
if 'true' in response.lower(): return 1.0
else: return 0.0
```

Deliberately kept unchanged for this task — it's what the LaRA paper reports, so it stays comparable to published numbers. Its weakness (documented separately for the professor's other ask): a correct-but-reworded answer often reads as "False" to a strict judge, collapsing partial credit to zero.

## Numeric judge: 0–10 rubric

A second, independent judge script (`compute_score_numeric.py`) — already built for the professor's numerical-scale ask, reused here unchanged.

```
RUBRIC = """Score the AI assistant's answer against the groundtruth on a 0-10 scale:  
- 10: fully correct and complete (rephrasing is fine).  
- 7-9: correct on the core point(s), missing only minor detail.  
- 4-6: partially correct; gets some but not all key facts.  
- 1-3: mostly incorrect but contains a relevant fragment.  
- 0: completely incorrect, irrelevant, or contradicts the groundtruth."""  
  
# a derived binary verdict keeps it comparable to the old scale:  
binary_verdict = (numeric_score >= BINARY_THRESHOLD) # default threshold = 7
```

Same generator answers, same questions, graded by the same 14B judge model — only the scale and prompt differ. This is what lets slide 12 show binary and numeric disagreeing on the identical answer.

# Same answer, same judge model, two verdicts

## QUESTION (REASONING · BOOK · SENTENCE CHUNKER)

**What lesson does Hartman learn from observing Brimstone's Christmas Eve party?**

## GROUNDTRUTH

Brimstone's generosity comes from simple gestures, not money; employers can make employees' lives joyful or miserable at low material cost.

## MODEL'S ANSWER

"Hartman learns the importance of making an effort for others and participating in community traditions, as evidenced by Brimstone's enthusiastic celebration..."

## NUMERIC JUDGE'S OWN REASONING

"Mentions participation in community traditions but misses the key point about simple gestures and employer influence."

**False**

binary judge (0.0)

**4 / 10**

numeric judge — partial credit

The binary judge can only say the answer is wrong. The numeric judge says **why** it's wrong and **how** wrong — it captured the general shape of the answer but missed the specific lesson. That distinction is invisible on a 0/1 scale and is exactly the partial-credit signal the professor asked for.

# Scope of this run

## Fixed across all cells

- Generator: **qwen2.5:7b** · Judge: **qwen2.5:14b** (decoupled, via Ollama)
- Context length: **32k** only (chunking behavior is what's under test, not long-context capacity)
- chunk\_size=600 / overlap=100 (where applicable), held constant across all 5 strategies
- Retrieval: hybrid vector+BM25, top-5 after rerank — identical for every chunker

## Varied

- **5 chunkers**: token, sentence, recursive, semantic, fast
- **3 doc types**: book, financial, paper
- **4 task types**: location, reasoning, comp, hallu
- **5 questions** per (doc type × task type) cell — a bounded first pass, not the full 2,326-question benchmark

**60 unique questions × 5 chunkers = 300 generations, judged twice (binary + numeric) = 600 judge calls. ~38 minutes end to end.**

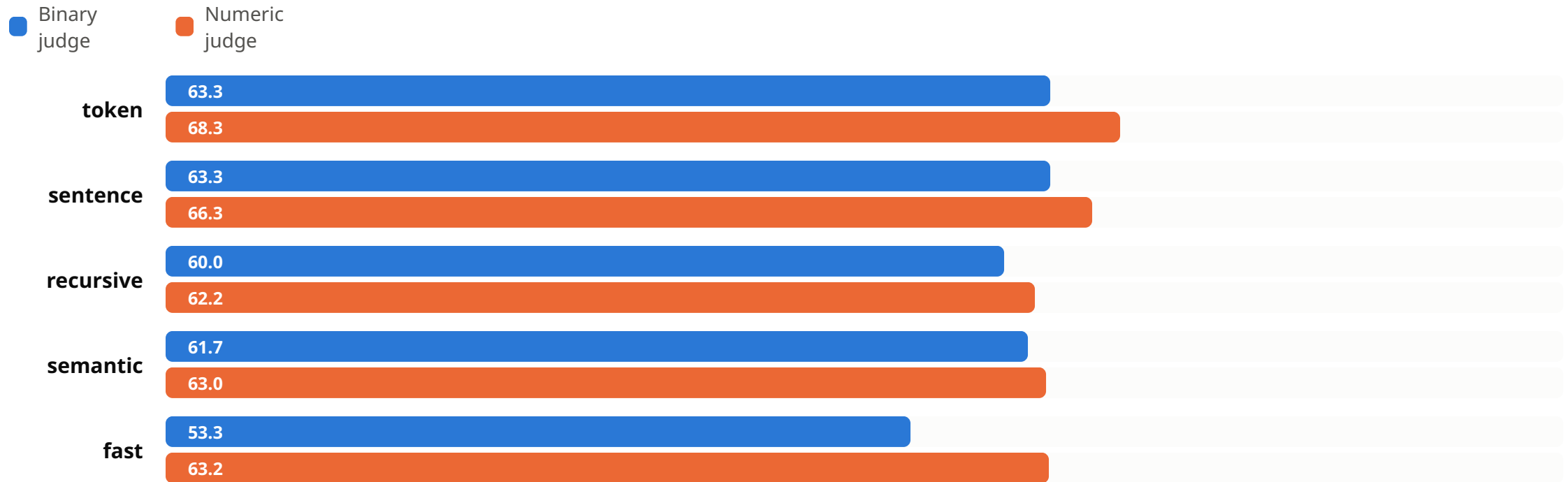
## RESULTS

# Accuracy by chunker and task type (%)

| CHUNKER   | LOCATION<br>BIN / NUM | REASONING<br>BIN / NUM | COMP<br>BIN / NUM | HALLU<br>BIN / NUM | OVERALL<br>BIN / NUM |
|-----------|-----------------------|------------------------|-------------------|--------------------|----------------------|
| token     | 66.7 / 82.7           | 53.3 / 58.0            | 53.3 / 56.0       | 80.0 / 76.7        | <b>63.3 / 68.3</b>   |
| sentence  | 66.7 / 76.7           | 46.7 / 50.7            | 40.0 / 56.7       | 100.0 / 81.3       | <b>63.3 / 66.3</b>   |
| recursive | 53.3 / 71.3           | 60.0 / 49.3            | 33.3 / 51.3       | 93.3 / 76.7        | <b>60.0 / 62.2</b>   |
| semantic  | 60.0 / 62.7           | 53.3 / 57.3            | 46.7 / 50.0       | 86.7 / 82.0        | <b>61.7 / 63.0</b>   |
| fast      | 60.0 / 75.3           | 40.0 / 44.7            | 20.0 / 48.7       | 93.3 / 84.0        | <b>53.3 / 63.2</b>   |

n=5 questions per cell — directional, not statistically tight (see limitations). "bin" = binary judge accuracy; "num"  
= numeric judge's mean score, normalized to a percentage.

## Binary vs numeric accuracy, by chunker



**fast** shows the widest gap between scales (53.3 binary vs 63.2 numeric) — its answers get marked wrong more often on the strict 0/1 scale than their actual content quality warrants. That gap is the exact phenomenon the numeric scale exists to surface.

# Three takeaways

- 1 **Token chunking ranks first** on both scales (63.3 binary / 68.3 numeric overall) in this bounded run — the simplest strategy, not the most sophisticated one, retrieved the most useful context for the LLM.
- 2 **Fast chunking ranks last on binary but not on numeric** (53.3 vs 63.2) — the same judge-collapse problem already flagged for the professor's other ask shows up again here, strategy-to-strategy, not just model-to-model.
- 3 Differences between strategies (53–63% binary, 62–68% numeric) are **real but not large** at this sample size — enough to show the machinery works and to motivate a wider run, not enough to declare a definitive winner yet.

# What this run does and doesn't show

## Scope limits

- **n=5** questions per cell (60 unique questions total) — a bounded first pass, chosen to fit a "this week" turnaround, not the full 2,326-question benchmark.
- **32k only** — no long-context comparison in this run.
- `chunk_size` held at 600 for comparability; a chunk-size sweep per strategy is a natural follow-up.

## The numeric scale isn't perfect either

- Example from this run (fast/comp): an answer that **misattributed a character name** still scored **7/10** → **"True"** at the default threshold — the judge rewarded overall shape over one factual error.
- Threshold=7 for the derived binary is a choice, not a law; different thresholds shift which strategies look best.
- Both scales share the same underlying judge model (qwen2.5:14b) — agreement between scales bounds internal consistency, not ground-truth correctness.



## REPRODUCING THIS

# One command, plus where the outputs land

```
LARA_LIMIT=5 ./run_chunker_study.sh
# override scope, e.g. more examples or fewer chunkers:
LARA_LIMIT=30 CHUNKERS="token semantic" ./run_chunker_study.sh
```

| OUTPUT                         | PATH                                                       |
|--------------------------------|------------------------------------------------------------|
| Per-question logs, both scales | prediction/result/combined_judgments_gen-*_chunker-*.jsonl |
| Per-chunker binary accuracy    | prediction/result/gen-*_chunker-*_all.csv                  |
| Per-chunker numeric scores     | prediction/result/numeric_gen-*_chunker-*_all.csv          |
| Final comparison table         | prediction/result/chunker_comparison.csv                   |

Next step, pending professor feedback: widen LARA\_LIMIT for a statistically firmer ranking, and/or sweep chunk\_size per strategy.